# Map Mapping Markers

## 1. Course Content

### Course Overview

After completing Cartographer sandbox mapping and route network annotation, we have a fine sandbox digital map. But in the computer's view, the map itself is just a bunch of laser point arrays and obstacle boundaries - it doesn't understand where is "shipping area", where is "Shelf 1", or where is "food waste garbage bin".

**Semantic Map Mapping**'s core mission is to build a bridge between the human's natural language world and the robot's mathematical space coordinates:

- **Human Perspective**: Natural language instructions just say *"Go to shipping area"*, *"Go to Shelf 3"*;
- **Robot Perspective**: After receiving instructions, quickly look up the corresponding global world coordinates

  $$(X, Y, Z)$$

  and quaternion heading

  $$(Q_x, Q_y, Q_z, Q_w)$$

  in the `map_mapping.yaml` mapping dictionary
- **Hand-Eye Coordination Perspective**: The parking position and car heading must be just right, ensuring after the cart stops, the onboard RGB-D camera can completely overlook and capture the bricks or shelf trays on the workbench!

### Learning Objectives

1. Understand the bridging role of semantic location mapping in embodied intelligence long-task planning.
2. Skilled use of `robot_control` CLI tool for high-precision coordinate marking of various workstations in the sandbox.
3. Master parking position and camera angle placement specifications for typical workstations (shelves, shipping tables, classified garbage bins).
4. Master the structure of `map_mapping.yaml` configuration file and manual fine-tuning techniques.

---

## 2. Preparation

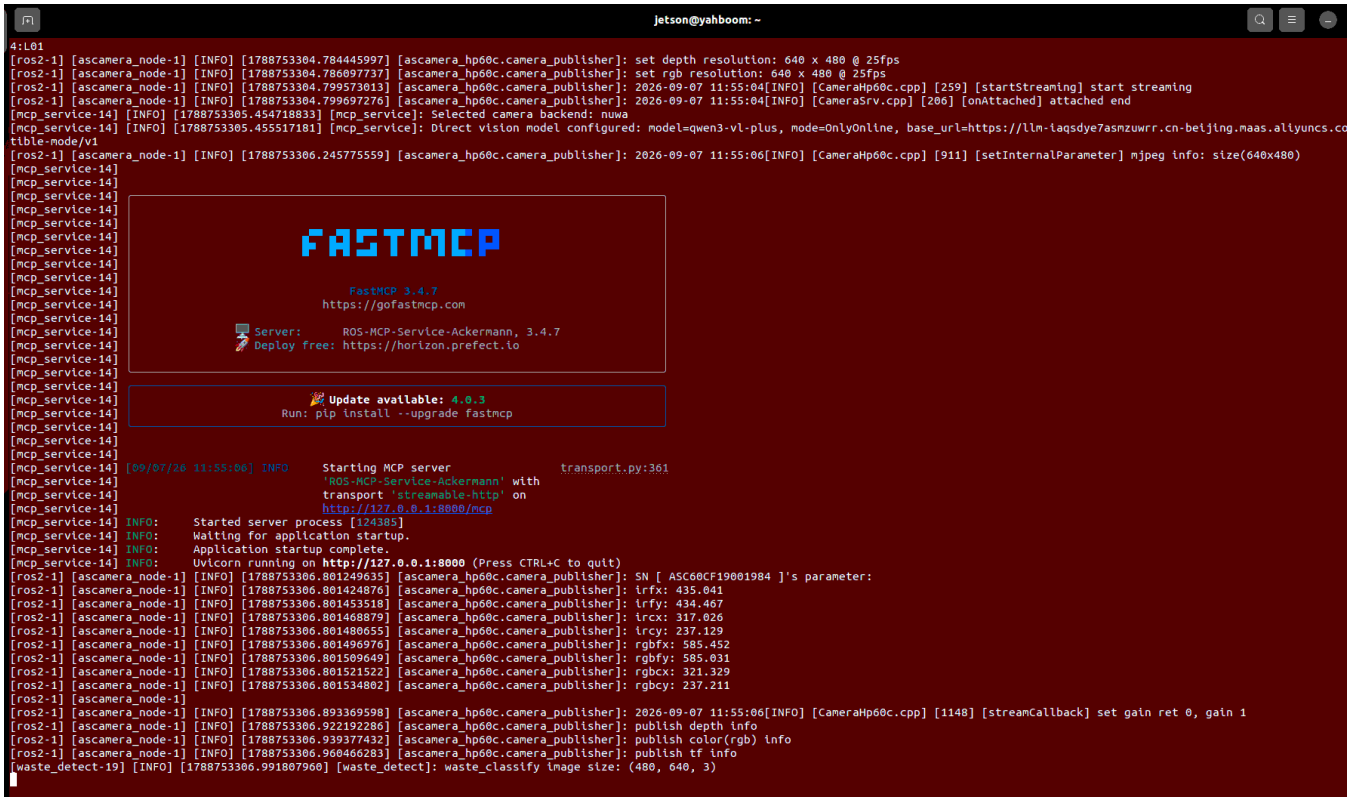
### 2.1 Complete Prerequisites

Please ensure you have completed [Build Sand Table Map] (having `map.yaml` and Cartographer pose graph `map.pbstream`) and [Visual Relocation Markers] (key workstation tags have been recorded through check-in tool).

## 2.2 Start MCP Core Service Node

Run the following startup command in the host terminal to pull up chassis drive, robotic arm assistance, TF transform, camera, and MCP scheduling service at once:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py relocation:=true
display_windows_only_track:=false
```



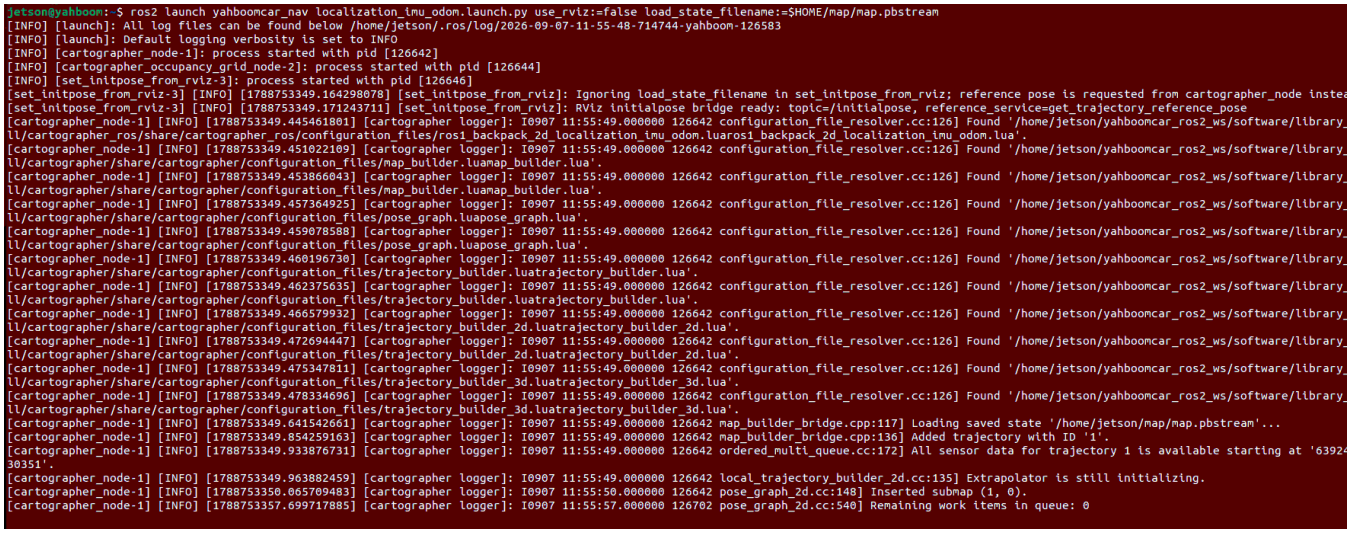

## 2.4 Start Cartographer Pose Graph Global Localization

Execute the following command in a new host terminal to load sandbox pose graph and start global localization:

```
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/jetson/map/map.pbstream

#pi5
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/root/map/map.pbstream

#rdk
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/sunrise/map/map.pbstream
```



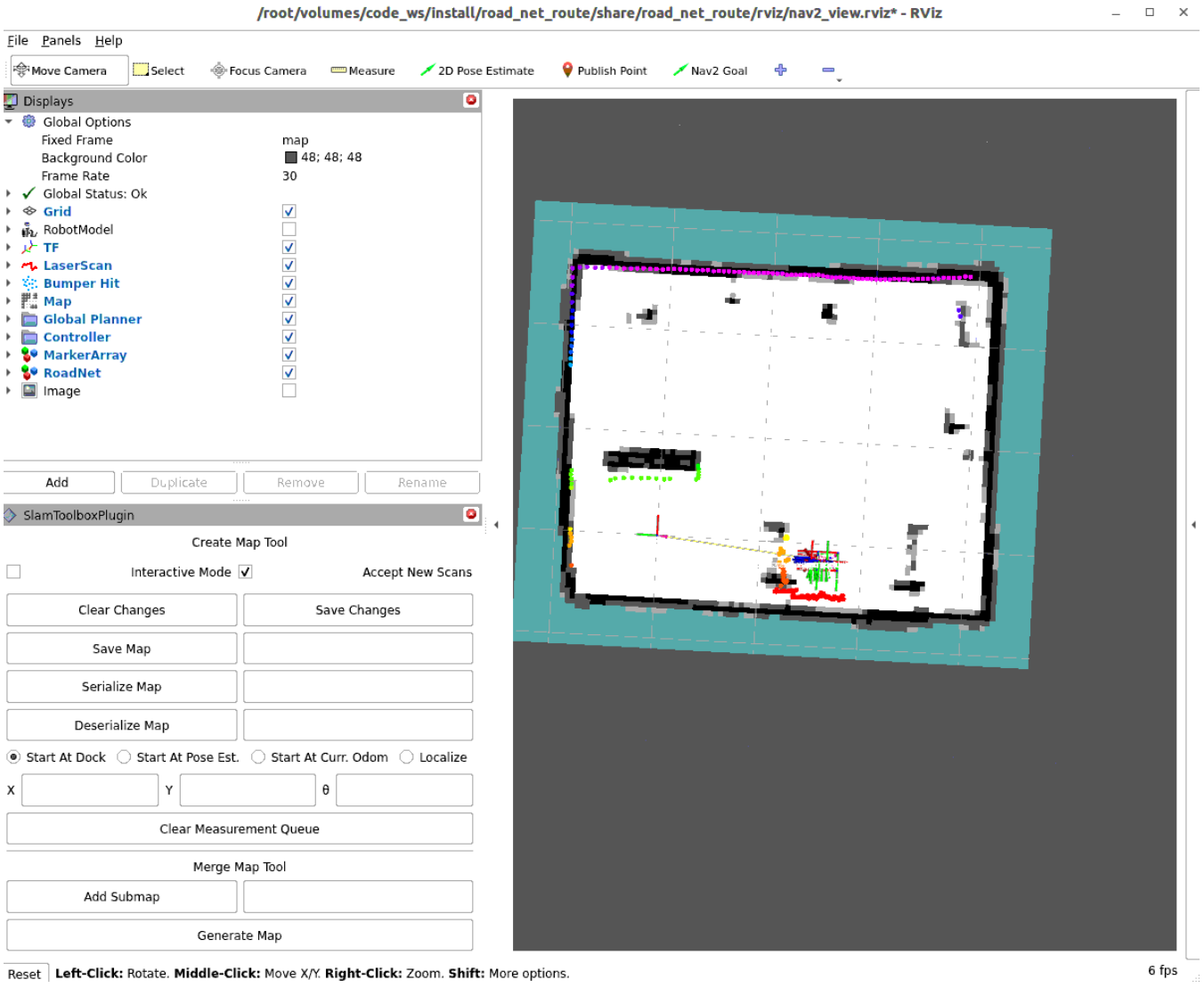

## 2.5 Start Route Network Docker Container and Route Network Navigation Service

Open a new terminal, enter the route network Docker container environment and start route network navigation:

```
# Quick start command
bringup_ros2kilted
# Enter container terminal
exec_ros2kilted
```

Start the visualization monitoring tool in the new route network terminal: **(RDK and Raspberry Pi need to run on virtual machine)**

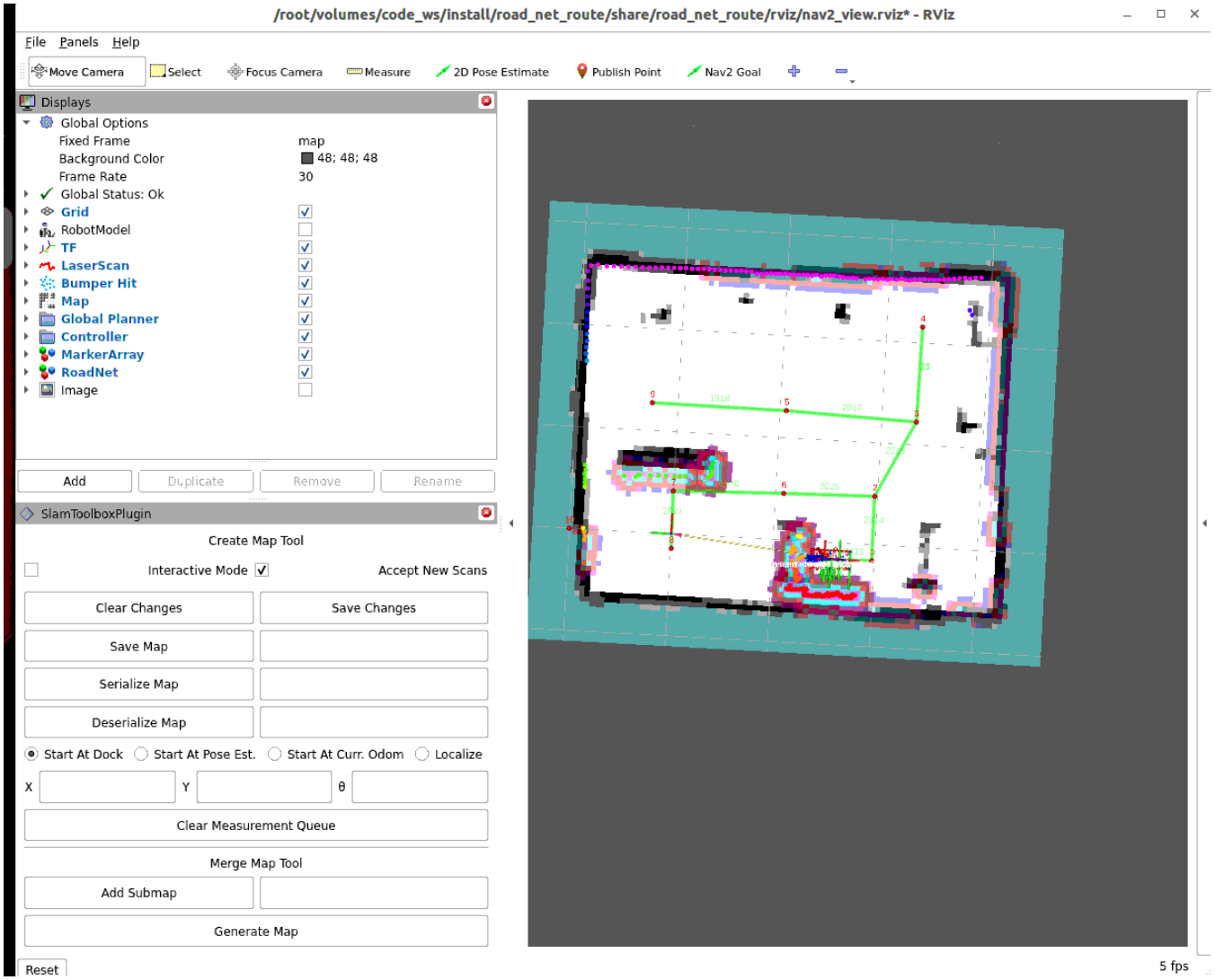
```
ros2 launch road_net_route rviz_only.launch.py
```




Start the route network navigation core node in the container:

```
ros2 launch road_net_route roadnet_nav2.launch.py
```

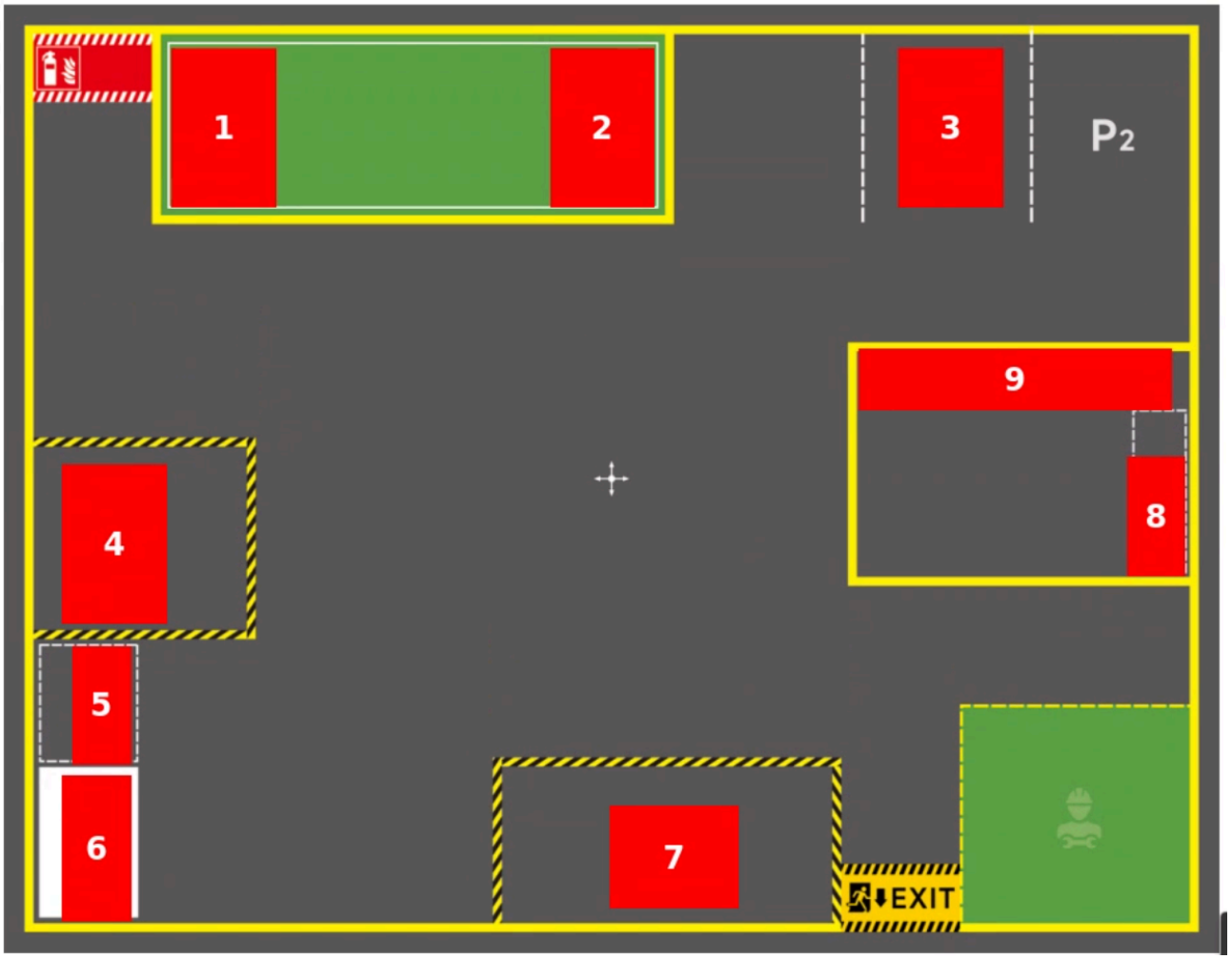
```
root@yahboom:~/volumes/code_ws# ros2 launch road_net_route roadnet_nav2.launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2026-09-07-03-56-19-166351-yahboom-149
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [launch.user]: Nav2 road network navigation start with external map->odom localization
[INFO] [route_bridge-2]: process started with pid [162]
[INFO] [component_container_isolated-1]: process started with pid [160]
[INFO] [launch_ros.actions.load_composable_nodes]: Loaded node '/controller_server' in container '/nav2_container'
[component_container_isolated-1] [INFO] [1788753381.478429195] [nav2_container]: Load Library: /root/navigation2/install/nav2_smoother/lib/libsmoother_server_core.so
[component_container_isolated-1] [INFO] [1788753381.621841431] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate<nav2_smoother::SmootherServer>
[component_container_isolated-1] [INFO] [1788753381.623235141] [nav2_container]: Instantiate class: rclcpp_components::NodeFactoryTemplate<nav2_smoother::SmootherServer>
[component_container_isolated-1] [INFO] [1788753381.683437389] [smoother_server]:
[component_container_isolated-1]        smoother_server lifecycle node launched.
[component_container_isolated-1]        Waiting on external lifecycle transitions to activate
[component_container_isolated-1]        See https://design.ros2.org/articles/node_lifecycle.html for more information.
[INFO] [launch_ros.actions.load_composable_nodes]: Loaded node '/smoother_server' in container '/nav2_container'
[component_container_isolated-1] [INFO] [1788753381.744800150] [smoother_server]: Creating smoother server
[component_container_isolated-1] [INFO] [1788753381.759276664] [nav2_container]: Load Library: /root/navigation2/install/nav2_planner/lib/libplanner_server_core.so
[component_container_isolated-1] [INFO] [1788753381.766168022] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate<nav2_planner::PlannerServer>
[component_container_isolated-1] [INFO] [1788753381.767455710] [nav2_container]: Instantiate class: rclcpp_components::NodeFactoryTemplate<nav2_planner::PlannerServer>
[component_container_isolated-1] [INFO] [1788753381.796135123] [planner_server]:
[component_container_isolated-1]        planner_server lifecycle node launched.
[component_container_isolated-1]        Waiting on external lifecycle transitions to activate
[component_container_isolated-1]        See https://design.ros2.org/articles/node_lifecycle.html for more information.
[component_container_isolated-1] [INFO] [1788753381.811914910] [planner_server]: Creating
[component_container_isolated-1] [INFO] [1788753381.845325753] [global_costmap.global_costmap]:
[component_container_isolated-1]        global_costmap lifecycle node launched.
[component_container_isolated-1]        Waiting on external lifecycle transitions to activate
[component_container_isolated-1]        See https://design.ros2.org/articles/node_lifecycle.html for more information.
[component_container_isolated-1] [INFO] [1788753381.863930305] [global_costmap.global_costmap]: Creating Costmap
[INFO] [launch_ros.actions.load_composable_nodes]: Loaded node '/planner_server' in container '/nav2_container'
[component_container_isolated-1] [INFO] [1788753381.889710901] [nav2_container]: Load Library: /root/navigation2/install/nav2_route/lib/libroute_server_core.so
[component_container_isolated-1] [INFO] [1788753381.902223211] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate<nav2_route::RouteServer>
[component_container_isolated-1] [INFO] [1788753381.903819107] [nav2_container]: Instantiate class: rclcpp_components::NodeFactoryTemplate<nav2_route::RouteServer>
[component_container_isolated-1] [INFO] [1788753381.936082815] [route_server]:
[component_container_isolated-1]        route_server lifecycle node launched.
[INFO] [launch_ros.actions.load_composable_nodes]: Loaded node '/route_server' in container '/nav2_container'
[component_container_isolated-1]        Waiting on external lifecycle transitions to activate
[component_container_isolated-1]        See https://design.ros2.org/articles/node_lifecycle.html for more information.
[component_container_isolated-1] [INFO] [1788753381.965447259] [nav2_container]: Load Library: /root/navigation2/install/nav2_behaviors/lib/libbehavior_server_core.so
[component_container_isolated-1] [INFO] [1788753381.968400767] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate<behavior_server::BehaviorServer>
[component_container_isolated-1] [INFO] [1788753381.968520645] [nav2_container]: Instantiate class: rclcpp_components::NodeFactoryTemplate<behavior_server::BehaviorServer>
```




# 3. Sandbox Key Workstation Planning and Marking Practice

Workstation names within the sandbox support free customization. The following is the factory-recommended standard practical function point list:

1. **Shelf 1** (Identifier: A)
2. **Shelf 2** (Identifier: B)
3. **Parking Lot 1** (Identifier: P1, robot departure and return)
4. **Goods Warehouse** (Identifier: C)
5. **Goods Sorting Area** (Identifier: D)
6. **Shelf 3** (Identifier: E)
7. **Shipping Area** (Identifier: F)
8. **Garbage Sorting Area** (Identifier: G)
9. **Garbage Bin Area**: Including **Food Waste Bin** (K), **Recyclable Bin** (R), **Other Waste Bin** (O), **Hazardous Waste Bin** (H)




## Marking

Execute marking in terminal using `robot_control` CLI tool:

```
robot_control call-tool RecordMapLocation --name C_Area --symbol A
```

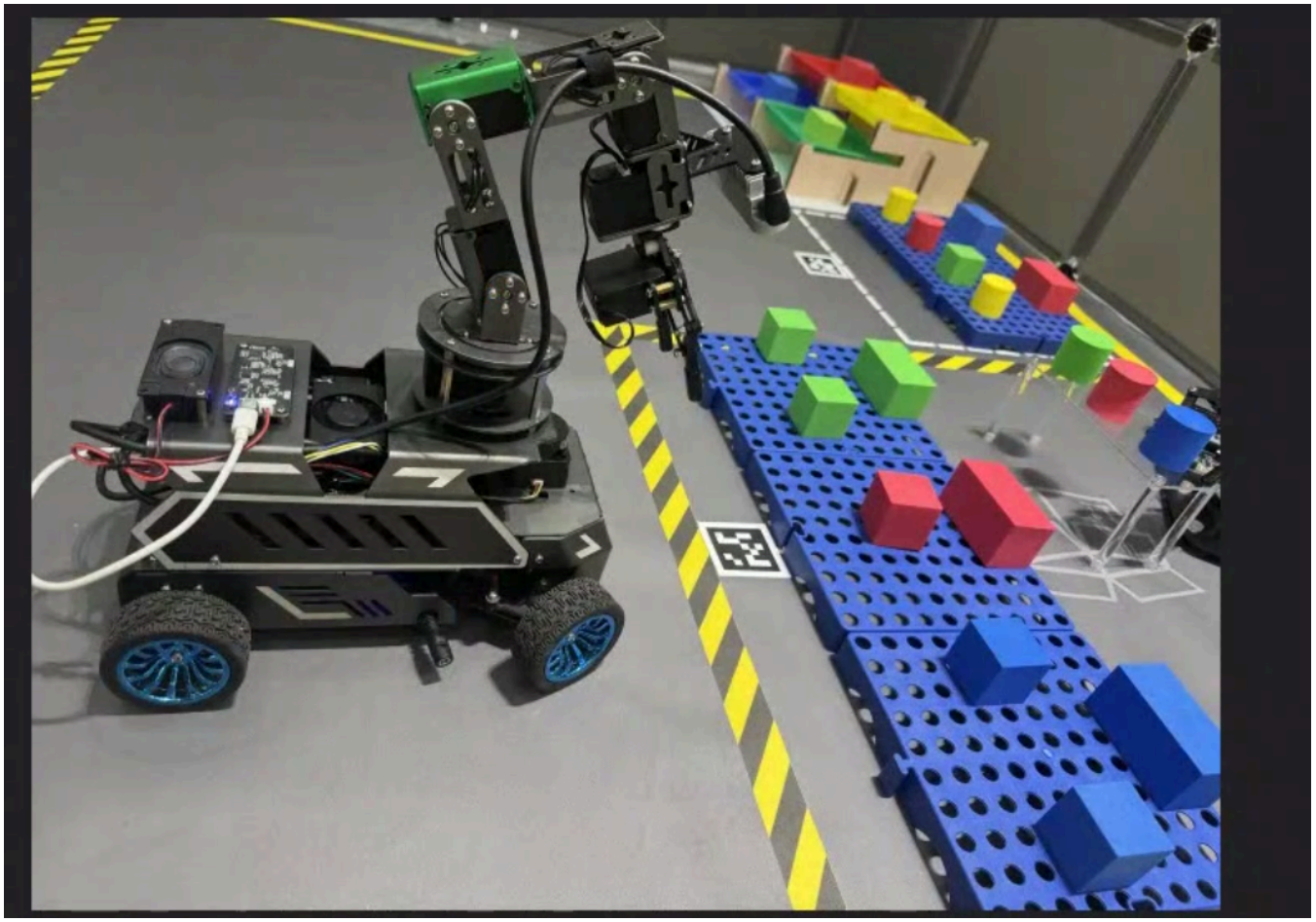
```
jetson@yahboom:~$ robot_control call-tool RecordMapLocation --name C□ --symbol A
{
  "execution_result": "success",
  "reason": "RecordMapLocation completed in 0.1s"
}
jetson@yahboom:~$
```

> **Important**
>
> **Marking Rules (Must Read to Avoid Pitfalls)**:
>
> 1. **Cart Head Facing**: The cart head must be vertically facing the material table or shelf, with the distance from cart head to table edge maintained between **25 ~ 35 cm**;
> 2. **Full Frame**: Before check-in, you can open the camera topic to view the current image, ensuring all brick cubes on the material table are in the middle lower half of the image without any obstruction;
> 3. **No Localization Drift**: Check whether the colored laser point cloud in RViz completely coincides with the black and white map outline. If there is slight deviation, first use the top `2D Pose Estimate` tool to align, then press Enter to execute recording!

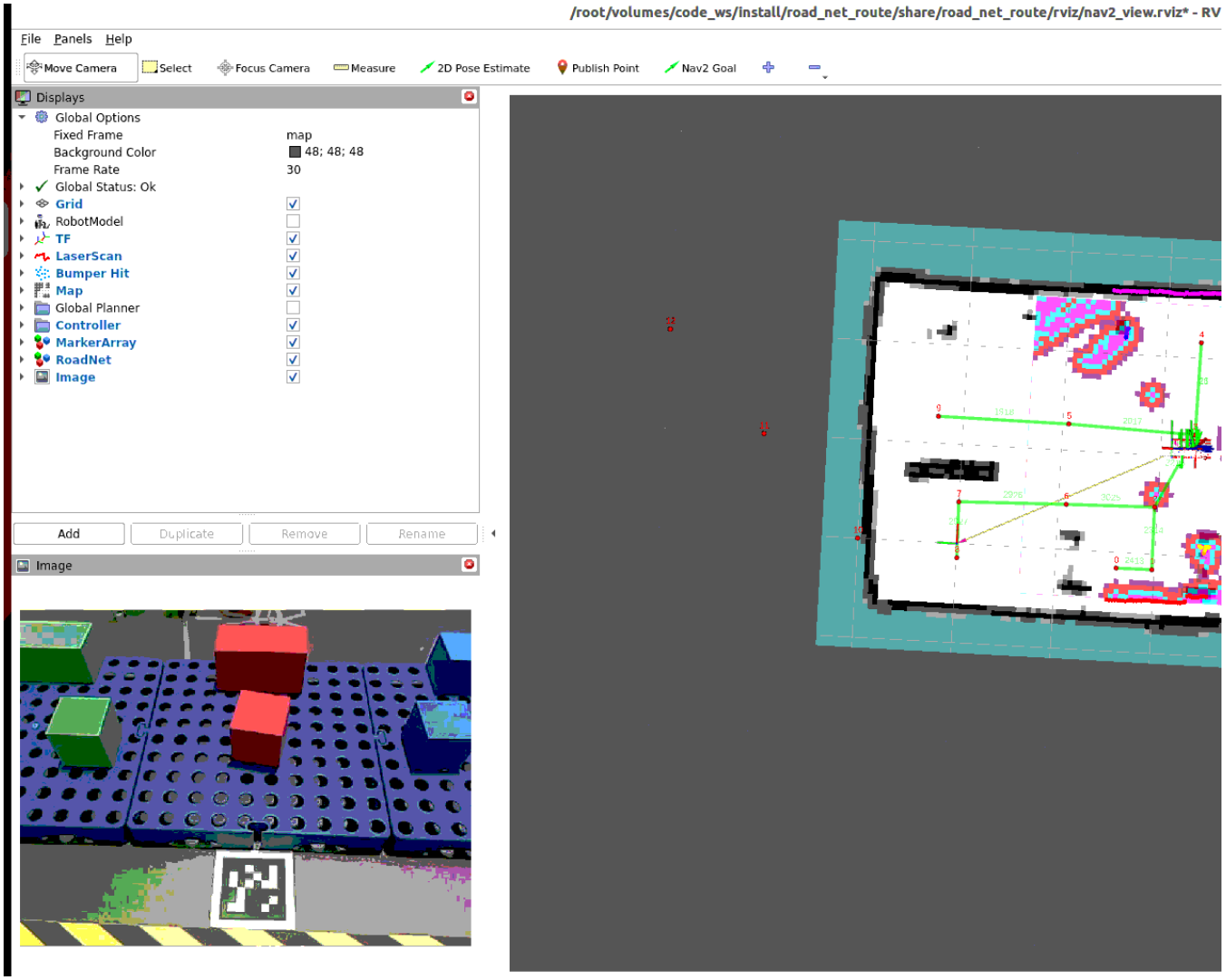
## Typical Workstation Marking Practice Demo

### Case 1: Marking "Shipping Area"

1. Remotely control the cart to the front of the shipping area, align the heading, with moderate distance:



2. Observe the onboard camera field of view, confirming that all cargo cubes on the shipping area mat can be completely and clearly captured:




3. Call the command in terminal to record the current cart `base_link` spatial pose to the map:

```
robot_control call-tool RecordMapLocation --name ShippingArea --symbol B
```

- `--name`: Chinese full name, when the LLM hears "Go to shipping area" it will match directly by this name;
- `--symbol`: Abbreviated code for quick indexing and command line debugging.

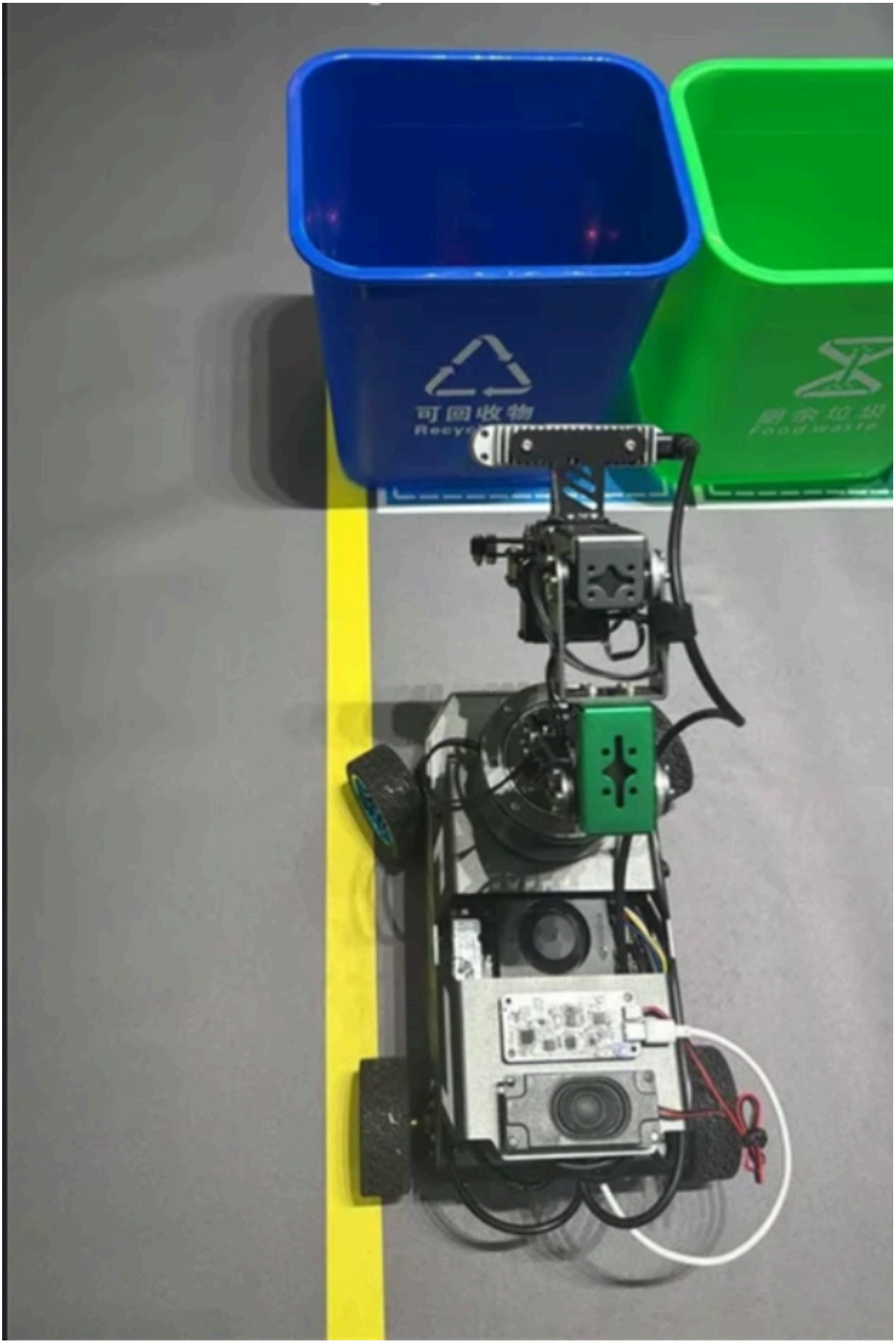
```
jetson@yahboom:~$ robot_control call-tool RecordMapLocation --name 发货区 --symbol B
{
  "execution_result": "success",
  "reason": "RecordMapLocation completed in 0.1s"
}
jetson@yahboom:~$
```

Terminal prints successful save and shows the captured world coordinate values in real-time.

```
mcp_service-14] INFO:     127.0.0.1:52766 - "POST /mcp HTTP/1.1" 200 OK
mcp_service-14] [INFO] [1788753960.052763011] [mcp_service]: Current location: x=1.08, y=-2.213, z=-0.005
mcp_service-14] [WARN] [1788753960.110833084] [mcp_service]: Symbol 'B' already exists, overwriting
mcp_service-14] [INFO] [1788753960.137320364] [mcp_service]: Location recorded successfully: B - 发货区
mcp_service-14] [INFO] [1788753960.147014625] [mcp_service]: MCP RecordMapLocation finished: ok=True, elapsed=0.1s
mcp_service-14] INFO:     127.0.0.1:52770 - "POST /mcp HTTP/1.1" 200 OK
mcp_service-14] INFO:     127.0.0.1:52786 - "DELETE /mcp HTTP/1.1" 200 OK
```

## Case 2: Marking Different Classified Garbage Bin Workstations

Since the four classified garbage bins are placed side by side, the robotic arm angles and cart parking positions are slightly different when delivering different garbage. It is recommended to drive the cart to the front of each garbage bin at approximately 25cm one by one for independent recording:




For example, recording food waste garbage bin:

- Later will find corresponding garbage bin based on garbage category:

- HazardousWaste → Hazardous waste

- FoodWaste → Food waste

- OtherWaste → Other waste

- Recyclablewaste→ Recyclable waste

```
robot_control call-tool RecordMapLocation --name Recyclablewaste --symbol K
```

---

## View and Verify Generated Map Mapping File

After all workstation marking is complete, data will automatically write to the following persistent configuration file:

```
~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_bringup/config/map_mapping.yaml
# Installation run directory sync at:
#~/yahboomcar_ros2_ws/dofbot_pro_ws/install/dofbot_pro_bringup/share/dofbot_pro_bringup/config/map_mapping.yaml
```

Run the following command to view file content:

```
cat
~/yahboomcar_ros2_ws/dofbot_pro_ws/install/dofbot_pro_bringup/share/dofbot_pro_bringup/config/map_mapping.yaml
```

Terminal will print the coordinate dictionary of all workstations in the entire sandbox:

```
jetson@yahboom:~$ cat ~/yahboomcar_ros2_ws/dofbot_pro_ws/install/dofbot_pro_bringup/share/dofbot_pro_bringup/config/map_mapping.yaml
A:
  name: C区
  position:
    x: 1.077
    y: -2.215
    z: -0.005
  orientation:
    x: 0.0
    y: 0.0
    z: -0.69
    w: 0.723
B:
  name: 发货区
  position:
    x: 1.08
    y: -2.213
    z: -0.005
  orientation:
    x: 0.0
    y: 0.0
    z: -0.69
    w: 0.724
```

```
K:
  name: Recyclablewaste
  position:
    x: 0.982
    y: -0.219
    z: -0.005
  orientation:
    x: 0.0
    y: 0.0
    z: 1.0
    w: 0.002
jetson@yahboom:~$
```

At this point, the entire sandbox map has been completely endowed with "human-readable, machine-interpretable" embodied spatial semantics, and the robot is fully ready to accept complex factory comprehensive task scheduling!